

4.3 CBAM 通道-空间注意力

借鉴 Woo 等人的 CBAM [4]。先对特征图做平均/最大池化生成通道描述，经共享两层 MLP 得到通道权重；再对通道维做平均/最大池化并拼接，经 7x7 卷积得到空间权重。两种注意力按顺序乘回特征图，最后再 GAP 分类。

$$F' = M_c(F) \otimes F; \quad F'' = M_s(F') \otimes F'$$

M_c 为通道注意力, M_s 为空间注意力, $\otimes$ 为逐元素乘法

实验结果 L4 相对 L2 的 overall 下降 0.05 点、全类 MPC 下降 1.19 点；当前没有证据支持额外注意力模块。

4.4 主干敏感性：ConvNeXt 与 ViT

14. **ConvNeXt-Tiny** [6]：现代纯卷积主干，27.85M 参数；用于检验结论是否依赖 ResNet50。
15. **ViT-B/16** [7]：将 448x448 图像切成 16x16 patch，共形成 28x28=784 个空间 token；代码把 token 还原为空间特征后统一接 GAP。
16. **结果**：两者与 L5 的 top-1/MPC 基本打平，但 ViT 86.28M 参数、50.4 s/epoch，计算代价最高。

5. 长尾损失、训练策略与评价指标

5.1 Class-Balanced Cross-Entropy

借鉴 Cui 等人的 Effective Number 思想 [5]。设第 y 类训练样本数为 n_y ， $\beta=0.9999$ ，先计算有效样本数，再用其倒数作为类别权重并归一化到平均权重为 1。

$$E(n_y) = (1 - \beta^{n_y}) / (1 - \beta); \quad w_y \propto 1 / E(n_y)$$

β 越接近 1，权重越接近逆频率；归一化保持总体损失尺度稳定

实现中的关键修正：PyTorch 的 `CrossEntropyLoss(weight=..., label_smoothing=0.1)` 会把稀有类权重施加到平滑目标中的所有类别项。长尾极端时，这会让每个样本都收到巨大的稀有类惩罚。当前代码先计算逐样本平滑 CE，再仅用真实标签对应的 w_y 加权。